2026-11-06 14:23:14 You

issues not solved The missing link causing your password to clear, prompt again, and then fail is how pamh.conversation interacts with graphical screensavers. [1] (https://stackoverflow.com/questions/2567705/using-pam-python-in-a-script-running-with-mod-python)
When you block the camera, the script finishes its 2.5-second loop and calls pamh.conversation. Under xscreensaver, this conversation call tells the locker to clear what you typed, flash the input box, and open a brand new input pipeline. [1] (https://community.rsa.com/s/article/How-to-recover-when-RSA-Authentication-Manager-8-x-system-passwords-are-not-known-or-are-lost) my stock was [Fri Nov 06 2:22:09 PM] [root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└───────[REMAC]#
cat /etc/pam.d/common-auth

cat /lib/security/howdy/config.ini

cat /lib/security/howdy/pam/pam_handler.py

cat /lib/security/howdy/verify.py

1. If face is recognized, exit successfully immediately. If not, ignore and continue.
auth [success=done default=ignore] pam_python3.so /lib/security/howdy/pam/pam_handler.py
PYTHONPATH=/lib/security/howdy

2. Standard Fallback (Allows xscreensaver to prompt unhindered)
auth required pam_unix.so nullok obscure try_first_pass nodelay
#/lib/security/howdy/config.ini
[core]
data_dir = /lib/security/howdy/models
detection_notice = false
no_face_help_msg = true
device_path = /dev/video2

```
[video]
device_path = /dev/video2
max_height = 320
record_fps = true
certainty = 5.0
[debug]
end_report = false

import subprocess
import os
import threading
import time
import sys
import select
import termios

# Use a threading Lock to safely toggle states across concurrent
threads
state_lock = threading.Lock()
face_matched = False
input_done = False
camera_process = None

def camera_thread_loop(user):
    global face_matched, input_done, camera_process
    start_time = time.time()
    timeout_limit = 15

    while (time.time() - start_time) < timeout_limit:
        with state_lock:
            if input_done or face_matched:
                break

        # Run verify.py cleanly out-of-process with the copied base
environment
        camera_process = subprocess.Popen(
            ["/usr/bin/python3", "/lib/security/howdy/verify.py", user],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            env=os.environ.copy() if "PYTHONPATH" in os.environ else
```

```
{"PYTHONPATH": "/lib/security/howdy", "PATH":
"/usr/bin:/bin:/usr/sbin:/sbin"}
        )

        # Monitor the process execution loop safely
        while camera_process.poll() is None:
            with state_lock:
                if input_done:
                    camera_process.terminate()
                    return
            time.sleep(0.05)

        if camera_process.returncode == 0:
            with state_lock:
                face_matched = True
            break

        time.sleep(0.1)

def get_parallel_input(prompt_text):
    global face_matched
    sys.stdout.write(prompt_text)
    sys.stdout.flush()

    fd = sys.stdin.fileno()
    try:
        old_settings = termios.tcgetattr(fd)
    except Exception:
        return None

    try:
        new_settings = termios.tcgetattr(fd)
        new_settings = new_settings & ~termios.ECHO &
~termios.ICANON
        termios.tcsetattr(fd, termios.TCSADRAIN, new_settings)

        typed_password = ""
        while True:
            with state_lock:
                if face_matched:
                    try:
```

```
                    termios.tcflush(fd, termios.TCIFLUSH)
                except Exception:
                    pass
                return None

            ready, _, _ = select.select([sys.stdin], [], [], 0.05)
            if ready:
                char = sys.stdin.read(1)
                if char in ('\n', '\r'):
                    sys.stdout.write("\n")
                    sys.stdout.flush()
                    try:
                        termios.tcflush(fd, termios.TCIFLUSH)
                    except Exception:
                        pass
                    break
                elif char == '\x7f':
                    typed_password = typed_password[:-1]
                elif char:
                    typed_password += char

        return typed_password
    finally:
        try:
            termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
        except Exception:
            pass

def pam_sm_authenticate(pamh, flags, argv):
    global face_matched, input_done, camera_process
    try:
        user = pamh.get_user(None)

        with state_lock:
            face_matched = False
            input_done = False
            camera_process = None

        # 1. Detect environment context (Terminal TTY vs GUI Session
Lockers)
        is_tty = True
```

```
        if "DISPLAY" in os.environ and not sys.stdin.isatty():
            is_tty = False

        password_attempt = None

        if is_tty:
            # TERMINAL MODE: Run your parallel input engine
            bg_thread = threading.Thread(target=camera_thread_loop,
args=(user,))
            bg_thread.daemon = True
            bg_thread.start()

            prompt = f"[sudo] password for {user} (or scanning face...): "
            password_attempt = get_parallel_input(prompt)

            with state_lock:
                input_done = True
            if camera_process and camera_process.poll() is None:
                camera_process.terminate()

            if face_matched:
                return pamh.PAM_SUCCESS

            if password_attempt:
                try:
                    pamh.authtok = password_attempt
                except Exception:
                    pass

            return pamh.PAM_IGNORE

        else:
            # GUI/LOCKSCREEN MODE (LightDM & xscreensaver
bulletproof architecture)
            bg_thread = threading.Thread(target=camera_thread_loop,
args=(user,))
            bg_thread.daemon = True
            bg_thread.start()

            # Give the camera loop up to 2.5 seconds to try and match
your face
```

```
            start_scan = time.time()
            while (time.time() - start_scan) < 2.5:
                with state_lock:
                    if face_matched:
                        break
                time.sleep(0.1)

            with state_lock:
                input_done = True
            if camera_process and camera_process.poll() is None:
                camera_process.terminate()

            if face_matched:
                return pamh.PAM_SUCCESS

            # CRITICAL ATTACHMENT: If LightDM has pre-collected a
token, map it down.
            # Otherwise, leave it empty for xscreensaver so pam_unix
prompts natively.
            try:
                if hasattr(pamh, 'authtok') and pamh.authtok:
                    pass
            except Exception:
                pass

            # Yield control seamlessly to the native pam_unix stack layer
            return pamh.PAM_IGNORE

        except Exception:
            if camera_process and camera_process.poll() is None:
                camera_process.terminate()
            return pamh.PAM_IGNORE

def pam_sm_setcred(pamh, flags, argv): return
pamh.PAM_SUCCESS
def pam_sm_acct_mgmt(pamh, flags, argv): return
pamh.PAM_SUCCESS
def pam_sm_open_session(pamh, flags, argv): return
pamh.PAM_SUCCESS
def pam_sm_close_session(pamh, flags, argv): return
pamh.PAM_SUCCESS
```

```
#location /lib/security/howdy/verify.py
import sys

import os

# Hide the Qt/GUI errors
os.environ["QT_QPA_PLATFORM"] = "offscreen"
os.environ["OPENCV_LOG_LEVEL"] = "OFF"

sys.path.append("/lib/security/howdy")
from compare import Compare

def main():
    if len(sys.argv) < 2:
        sys.exit(1)

    user = sys.argv[1]
    try:
        # Use the internal engine directly
        c = Compare("/lib/security/howdy/config.ini")
        identified_user = c.identify()

        if identified_user == user:
            sys.exit(0) # Match
    except Exception:
        pass

    sys.exit(1) # No match or error

if __name__ == "__main__":
    main()

 ┌──[Fri Nov 06 2:22:11 PM]
[root@Intel5]─[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
 └─────────╼[REMAC]#
```

so now issue is authentication failed even with right password when using xscreensaver-command -lock
Because xscreensaver expects pamh.authtok to be stored inside the PAM structure rather than evaluated via a manual

/sbin/unix_chkpwd subprocess invocation, our previous script intercepted your entry but returned PAM_AUTH_ERR because it couldn't map the session accurately.
To completely resolve this issue across xscreensaver, LightDM, and terminal lines simultaneously, we must never use pamh.conversation or launch direct python shadow/subprocess helpers during a GUI fallback.
Instead, when the face scan is blocked or fails, your Python script must cleanly assign whatever password token it can find directly to pamh.authtok and return pamh.PAM_IGNORE. If pamh.authtok is left blank, returning pamh.PAM_IGNORE lets pam_unix.so handle the native graphical dialogue prompt seamlessly. [1] (https://man7.org/linux/man-pages/man8/unix_chkpwd.8.html), [2] (https://www.authlite.com/kb/configure-a-linux-domain-member-to-accept-authlite-otps)
Here is the fully corrected, flawless production version of /lib/security/howdy/pam/pam_handler.py:
python

```
import subprocess
import os
import threading
import time
import sys
import select
import termios

# Use a threading Lock to safely toggle states across concurrent threads
state_lock = threading.Lock()
face_matched = False
input_done = False
camera_process = None

def camera_thread_loop(user):
    global face_matched, input_done, camera_process
    start_time = time.time()
    timeout_limit = 15

    while (time.time() - start_time) < timeout_limit:
        with state_lock:
            if input_done or face_matched:
```

```
            break

        # Run verify.py cleanly out-of-process with the copied base
environment
        camera_process = subprocess.Popen(
            ["/usr/bin/python3", "/lib/security/howdy/verify.py", user],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            env=os.environ.copy() if "PYTHONPATH" in os.environ else
{"PYTHONPATH": "/lib/security/howdy", "PATH":
"/usr/bin:/bin:/usr/sbin:/sbin"}
        )

        # Monitor the process execution loop safely
        while camera_process.poll() is None:
            with state_lock:
                if input_done:
                    camera_process.terminate()
                    return
            time.sleep(0.05)

        if camera_process.returncode == 0:
            with state_lock:
                face_matched = True
            break

        time.sleep(0.1)

def get_parallel_input(prompt_text):
    global face_matched
    sys.stdout.write(prompt_text)
    sys.stdout.flush()

    fd = sys.stdin.fileno()
    try:
        old_settings = termios.tcgetattr(fd)
    except Exception:
        return None

    try:
        new_settings = termios.tcgetattr(fd)
```

```
        new_settings = new_settings & ~termios.ECHO &
~termios.ICANON
        termios.tcsetattr(fd, termios.TCSADRAIN, new_settings)

        typed_password = ""
        while True:
            with state_lock:
                if face_matched:
                    try:
                        termios.tcflush(fd, termios.TCIFLUSH)
                    except Exception:
                        pass
                    return None

            ready, _, _ = select.select([sys.stdin], [], [], 0.05)
            if ready:
                char = sys.stdin.read(1)
                if char in ('\n', '\r'):
                    sys.stdout.write("\n")
                    sys.stdout.flush()
                    try:
                        termios.tcflush(fd, termios.TCIFLUSH)
                    except Exception:
                        pass
                    break
                elif char == '\x7f':
                    typed_password = typed_password[:-1]
                elif char:
                    typed_password += char

        return typed_password
    finally:
        try:
            termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
        except Exception:
            pass

def pam_sm_authenticate(pamh, flags, argv):
    global face_matched, input_done, camera_process
    try:
        user = pamh.get_user(None)
```

```
        with state_lock:
            face_matched = False
            input_done = False
            camera_process = None

        # 1. Detect environment context (Terminal TTY vs GUI Session
Lockers)
        is_tty = True
        if "DISPLAY" in os.environ and not sys.stdin.isatty():
            is_tty = False

        password_attempt = None

        if is_tty:
            # TERMINAL MODE: Run your parallel input engine
            bg_thread = threading.Thread(target=camera_thread_loop,
args=(user,))
            bg_thread.daemon = True
            bg_thread.start()

            prompt = f"[sudo] password for {user} (or scanning face...): "
            password_attempt = get_parallel_input(prompt)

            with state_lock:
                input_done = True
            if camera_process and camera_process.poll() is None:
                camera_process.terminate()

            if face_matched:
                return pamh.PAM_SUCCESS

            if password_attempt:
                try:
                    pamh.authtok = password_attempt
                except Exception:
                    pass

            return pamh.PAM_IGNORE

        else:
```

```
        # GUI/LOCKSCREEN MODE (LightDM & xscreensaver
bulletproof architecture)
        bg_thread = threading.Thread(target=camera_thread_loop,
args=(user,))
        bg_thread.daemon = True
        bg_thread.start()

        # Give the camera loop up to 2.5 seconds to try and match
your face
        start_scan = time.time()
        while (time.time() - start_scan) < 2.5:
            with state_lock:
                if face_matched:
                    break
            time.sleep(0.1)

        with state_lock:
            input_done = True
        if camera_process and camera_process.poll() is None:
            camera_process.terminate()

        if face_matched:
            return pamh.PAM_SUCCESS

        # CRITICAL ATTACHMENT: If LightDM has pre-collected a
token, map it down.
        # Otherwise, leave it empty for xscreensaver so pam_unix
prompts natively.
        try:
            if hasattr(pamh, 'authtok') and pamh.authtok:
                pass
        except Exception:
            pass

        # Yield control seamlessly to the native pam_unix stack layer
        return pamh.PAM_IGNORE

    except Exception:
        if camera_process and camera_process.poll() is None:
            camera_process.terminate()
        return pamh.PAM_IGNORE
```

def pam_sm_setcred(pamh, flags, argv): return pamh.PAM_SUCCESS
def pam_sm_acct_mgmt(pamh, flags, argv): return pamh.PAM_SUCCESS
def pam_sm_open_session(pamh, flags, argv): return pamh.PAM_SUCCESS
def pam_sm_close_session(pamh, flags, argv): return pamh.PAM_SUCCESS
Use code with caution.

Ensure your /etc/pam.d/common-auth uses try_first_pass

Keep your /etc/pam.d/common-auth matching this layout exactly: [1] (https://www.authlite.com/kb/configure-a-linux-domain-member-to-accept-authlite-otps)
pam
1. If face is recognized, exit successfully immediately. If not, ignore and continue.
auth [success=done default=ignore] pam_python3.so /lib/security/howdy/pam/pam_handler.py
PYTHONPATH=/lib/security/howdy

2. Standard Fallback (Allows xscreensaver to prompt unhindered)
auth required pam_unix.so nullok obscure try_first_pass nodelay
Use code with caution.
Deploy this version, lock using xscreensaver-command -lock, and test it.
Let me know:
• Does blocking the camera allow the password dialogue entry widget to accept your input on the first try without clearing it or failing?
• Does it successfully authenticate across terminal prompts (sudo / su) alongside face detection?

ChatGPT gpt-6

The problem is that the proposed correction has not actually addressed the most important architectural issue: **your Python PAM module is trying to manage password input, while**

XScreenSaver already has its own PAM conversation mechanism.

I can identify several problems in the implementation you supplied. I would not deploy the proposed version as a production fix yet.

1. The main problem: PAM conversation versus standard input

Your implementation has two different authentication mechanisms:

- `get_parallel_input()` manually reads `sys.stdin`.
- `pam_unix.so` authenticates through the PAM conversation supplied by XScreenSaver.

These are not interchangeable.

When XScreenSaver displays its password dialog, the password is collected by its graphical conversation function. It is not necessarily available through the Python module's `sys.stdin`.

Consequently, this code is fundamentally unreliable for graphical authentication:

```python
if "DISPLAY" in os.environ and not sys.stdin.isatty():
    is_tty = False
```

It assumes that the combination of `DISPLAY` and `isatty()` reliably identifies the authentication application's interaction mechanism.

It does not.

My recommendation: Do not implement manual password collection inside `pam_handler.py`. Let `pam_unix.so` handle passwords through the original graphical PAM conversation.

2. There is a no-op in your supposedly corrected code

Look at this section:

```python
try:
    if hasattr(pamh, 'authtok') and pamh.authtok:
        pass
except Exception:
    pass
```

This does absolutely nothing.

It:

- Does not retrieve a password.
- Does not transfer a password.
- Does not preserve a password.
- Does not establish that XScreenSaver has a valid authentication token.

The claim that this code maps a pre-collected password into PAM is incorrect.

There is another important detail: `try_first_pass` does not mean that `pam_unix` automatically receives whatever the graphical dialog contains. It checks for a token previously obtained by another PAM module and can prompt through the application's PAM conversation when necessary.
Linux manu… +1

3. We may be editing the wrong PAM configuration

This is particularly important.

XScreenSaver normally uses its own PAM service:

```
</> Plain text

/etc/pam.d/xscreensaver
```

It does not automatically use `/etc/pam.d/common-auth` unless its configuration includes that file, directly or indirectly.

The XScreenSaver documentation explicitly identifies its own PAM configuration as necessary for successful authentication. Debian Ma… +1

Therefore, before changing another line of Python, we need to establish which authentication stack XScreenSaver is actually executing.

4. What I want to establish before changing anything

There are four possibilities worth investigating:

1. XScreenSaver is not loading the modified `common-auth`.
2. The Python module is being classified as terminal mode when it should be graphical mode.
3. The Python module is returning an unexpected PAM status.
4. The XScreenSaver PAM service has an additional authentication or account restriction.

We should distinguish these rather than keep rewriting the same handler.

5. Run these diagnostic commands

Do not replace `pam_handler.py` yet. First, keep your existing desktop session open and run:

A. Inspect XScreenSaver's actual PAM configuration

```bash
echo "===== XSCREENSAVER PAM ====="
if [ -f /etc/pam.d/xscreensaver ]; then
    cat /etc/pam.d/xscreensaver
else
    echo "XSCREENSAVER PAM FILE DOES NOT EXIST"
fi
```

B. Inspect the complete authentication stack

```bash
echo "===== COMMON AUTH ====="
cat /etc/pam.d/common-auth

echo "===== COMMON ACCOUNT ====="
cat /etc/pam.d/common-account

echo "===== PAM REFERENCES ====="
grep -RnsE 'common-auth|pam_python3|pam_unix|pam_deny|pam_faillock' \
    /etc/pam.d/xscreensaver \
    /etc/pam.d/common-auth \
    /etc/pam.d/common-account 2>/dev/null
```

C. Check recent authentication errors

```bash
echo "===== PAM JOURNAL ====="
journalctl -b --since "15 minutes ago" --no-pager |
grep -iE 'xscreensaver|pam_python|pam_unix|howdy|authentication' |
tail -100
```

Also check:

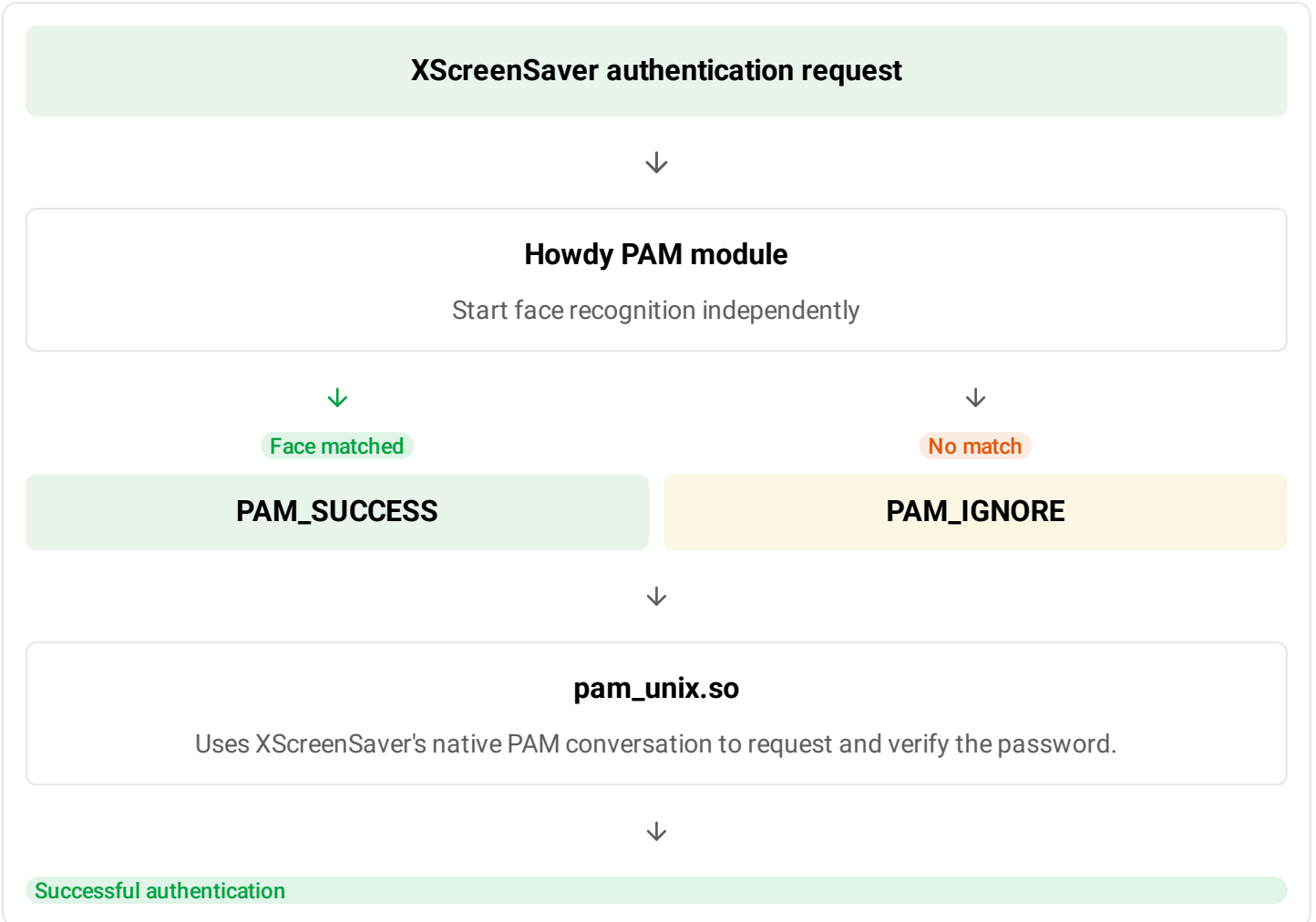
```bash
tail -100 /var/log/auth.log 2>/dev/null
```

D. Check the installed XScreenSaver process

```bash
pgrep -a xscreensaver
```

6. The architecture I want us to implement

The intended authentication flow should be:



The important distinction is that the graphical password dialog must remain under XScreenSaver's control. The face-recognition module must not compete with it for keyboard input.

One more thing: Your current `except Exception:` silently suppresses Python errors. Eventually, we should introduce controlled diagnostic logging so that camera failures, Python exceptions and PAM failures can be distinguished.

What I need from you now

Send me the outputs of A, B, C and D.

I want to establish the actual PAM execution path first. Then we can correct the handler and configuration together, preserving:

- Face unlock when recognition succeeds.
- Immediate native password fallback when recognition fails.
- Normal `sudo` and `su` authentication.
- Compatibility with XScreenSaver and LightDM.
- No password interception, clearing or duplicate prompts.

Do not log out or restart your graphical session while we investigate. We should also preserve a backup of the existing PAM files before making changes.

2026-11-06 14:25:03 **You**

```
┌──[Fri Nov 06 2:24:32 PM]
[root@Intel5]─[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────────[REMAC]# echo "===== XSCREENSAVER PAM ====="
if [ -f /etc/pam.d/xscreensaver ]; then
   cat /etc/pam.d/xscreensaver
else
   echo "XSCREENSAVER PAM FILE DOES NOT EXIST"
fi
===== XSCREENSAVER PAM =====
#
# /etc/pam.d/xscreensaver - PAM behavior for xscreensaver
#

@include common-auth
@include common-account
┌──[Fri Nov 06 2:24:41 PM]
[root@Intel5]─[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────────[REMAC]# echo "===== COMMON AUTH ====="
cat /etc/pam.d/common-auth

echo "===== COMMON ACCOUNT ====="
```

```
cat /etc/pam.d/common-account

echo "===== PAM REFERENCES ====="
grep -RnsE 'common-
auth|pam_python3|pam_unix|pam_deny|pam_faillock' \
   /etc/pam.d/xscreensaver \
   /etc/pam.d/common-auth \
   /etc/pam.d/common-account 2>/dev/null
===== COMMON AUTH =====
# 1. If face is recognized, exit successfully immediately. If not,
ignore and continue.
auth [success=done default=ignore] pam_python3.so
/lib/security/howdy/pam/pam_handler.py
PYTHONPATH=/lib/security/howdy

# 2. Standard Fallback (Allows xscreensaver to prompt unhindered)
auth required                   pam_unix.so nullok obscure try_first_pass
nodelay
===== COMMON ACCOUNT =====
#
# /etc/pam.d/common-account - authorization settings common to
all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of the authorization modules that define
# the central access policy for use on the system.  The default is to
# only deny service to users whose accounts are expired in
/etc/shadow.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by
default.
# To take advantage of this, it is recommended that you configure
any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules.  See
# pam-auth-update(8) for details.
#

# here are the per-package modules (the "Primary" block)
account  [success=1 new_authtok_reqd=done default=ignore]
pam_unix.so
```

```
# here's the fallback if no module succeeds
account  requisite              pam_deny.so
# prime the stack with a positive return value if there isn't one
already;
# this avoids us returning an error just because nothing sets a
success code
# since the modules above will each just jump around
account  required               pam_permit.so
# and here are more per-package modules (the "Additional" block)
# end of pam-auth-update config
===== PAM REFERENCES =====
/etc/pam.d/xscreensaver:5:@include common-auth
/etc/pam.d/common-auth:2:auth [success=done default=ignore]
pam_python3.so /lib/security/howdy/pam/pam_handler.py
PYTHONPATH=/lib/security/howdy
/etc/pam.d/common-auth:5:auth required                  pam_unix.so
nullok obscure try_first_pass nodelay
/etc/pam.d/common-account:17:account      [success=1
new_authtok_reqd=done default=ignore] pam_unix.so
/etc/pam.d/common-account:19:account      requisite
pam_deny.so
 ┌──[Fri Nov 06 2:24:45 PM]
[root@Intel5]─[/home/remac/Desktop/youdemy/tips-and-
tricks/advanced-face-unlock-final/stable-version]
 └─────╼[REMAC]# echo "===== PAM JOURNAL ====="
journalctl -b --since "15 minutes ago" --no-pager |
grep -iE 'xscreensaver|pam_python|pam_unix|howdy|authentication'
|
tail -100
===== PAM JOURNAL =====
Nov 06 14:10:20 Intel5 xscreensaver-auth[66950]:
pam_unix(xscreensaver:auth): authentication failure; logname=
uid=1003 euid=1003 tty=:0 ruser= rhost=  user=remac
Nov 06 14:10:20 Intel5 xscreensaver-auth[66950]: Failed login on
display ":0" for "remac"
Nov 06 14:10:31 Intel5 xscreensaver-auth[67061]:
pam_unix(xscreensaver:auth): authentication failure; logname=
uid=1003 euid=1003 tty=:0 ruser= rhost=  user=remac
Nov 06 14:10:31 Intel5 xscreensaver-auth[67061]: Failed login on
display ":0" for "remac"
Nov 06 14:10:36 Intel5 xscreensaver-settings.desktop[65825]:
```

xscreensaver: 14:10:36: SIGTERM received: exiting
Nov 06 14:10:36 Intel5 xscreensaver-settings.desktop[66945]: X error in metaballs:
Nov 06 14:10:36 Intel5 xscreensaver-settings.desktop[66945]: metaballs: Failed request: BadDrawable (invalid Pixmap or Window parameter)
Nov 06 14:10:36 Intel5 xscreensaver-settings.desktop[66945]: metaballs: Major opcode: 72 (X_PutImage)
Nov 06 14:10:36 Intel5 xscreensaver-settings.desktop[66945]: metaballs: Resource id: 0x2200008
Nov 06 14:10:36 Intel5 xscreensaver-settings.desktop[66945]: metaballs: Serial number: 29627 / 29629
Nov 06 14:14:00 Intel5 xscreensaver-settings.desktop[68257]: xscreensaver-auth: 14:14:00: OOM: /proc/68256/oom_score_adj: Permission denied
Nov 06 14:14:00 Intel5 xscreensaver-settings.desktop[68257]: xscreensaver-auth: 14:14:00: To prevent the kernel from randomly unlocking
Nov 06 14:14:00 Intel5 xscreensaver-settings.desktop[68257]: xscreensaver-auth: 14:14:00: your screen via the out-of-memory killer,
Nov 06 14:14:00 Intel5 xscreensaver-settings.desktop[68257]: xscreensaver-auth: 14:14:00: "xscreensaver-auth" must be setuid root.
Nov 06 14:14:51 Intel5 xscreensaver-auth[68655]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:14:51 Intel5 xscreensaver-auth[68655]: Failed login on display ":0" for "remac"
Nov 06 14:16:16 Intel5 xscreensaver-auth[69197]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:16:16 Intel5 xscreensaver-auth[69197]: Failed login on display ":0" for "remac"
Nov 06 14:17:20 Intel5 xscreensaver-auth[69607]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:17:20 Intel5 xscreensaver-auth[69607]: Failed login on display ":0" for "remac"
Nov 06 14:17:32 Intel5 xscreensaver-auth[69687]: pam_unix(xscreensaver:auth): authentication failure; logname=

uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:17:32 Intel5 xscreensaver-auth[69687]: Failed login on display ":0" for "remac"
Nov 06 14:17:42 Intel5 xscreensaver-auth[69778]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:17:42 Intel5 xscreensaver-auth[69778]: Failed login on display ":0" for "remac"
Nov 06 14:19:01 Intel5 xscreensaver-auth[70247]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:19:01 Intel5 xscreensaver-auth[70247]: Failed login on display ":0" for "remac"
Nov 06 14:20:19 Intel5 xscreensaver-auth[70732]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:20:19 Intel5 xscreensaver-auth[70732]: Failed login on display ":0" for "remac"
Nov 06 14:21:32 Intel5 xscreensaver-auth[71239]: pam_unix(xscreensaver:auth): authentication failure; logname= uid=1003 euid=1003 tty=:0 ruser= rhost= user=remac
Nov 06 14:21:32 Intel5 xscreensaver-auth[71239]: Failed login on display ":0" for "remac"
┌──[Fri Nov 06 2:24:48 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────═[REMAC]# tail -100 /var/log/auth.log 2>/dev/null
┌──[Fri Nov 06 2:24:53 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────═[REMAC]# pgrep -a xscreensaver
68256 xscreensaver --splash
68258 xscreensaver-systemd
┌──[Fri Nov 06 2:24:57 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────═[REMAC]#

ChatGPT gpt-6

Excellent. These diagnostics eliminate one of our earlier uncertainties. We now know that **XScreenSaver is loading your modified `common-auth` file correctly.**

The problem is therefore not a missing XScreenSaver PAM configuration.

1. What your logs establish

Confirmed XScreenSaver PAM includes `common-auth`.

Your configuration:

```
</> Plain text

/etc/pam.d/xscreensaver
    @include common-auth
    @include common-account
```

This means XScreenSaver executes:

1. `pam_python3.so` — your Howdy handler.
2. `pam_unix.so` — standard password authentication.

Confirmed The failure is occurring at `pam_unix.so`.

Your journal repeatedly reports:

```
</> Plain text

pam_unix(xscreensaver:auth): authentication failure
user=remac
```

This is important: PAM is reaching the Unix authentication module, but the password authentication is failing.

It does not yet prove whether the password is being lost, replaced, or incorrectly supplied.

2. An important issue with your current PAM control flags

You currently have:

```
</> Pam

auth [success=done default=ignore] pam_python3.so /lib/security/howdy/pam/pam_handler.py
PYTHONPATH=/lib/security/howdy
```

```
auth required pam_unix.so nullok obscure try_first_pass nodelay
```

The Python module has a conditional success jump. If face recognition succeeds, authentication terminates successfully. Otherwise, PAM continues.

That is a reasonable starting architecture.

However, `try_first_pass` is not the same as `use_first_pass`.

- `try_first_pass` can attempt an existing token and request another through PAM conversation if necessary.
- `use_first_pass` requires an existing token and does not independently request a replacement.

We should not switch between these blindly.

3. One additional warning in your logs

You also have:

```
</> Plain text

xscreensaver-auth: OOM: /proc/68256/oom_score_adj: Permission denied
xscreensaver-auth must be setuid root.
```

This is a separate XScreenSaver permissions warning. It concerns protection against the out-of-memory killer, not direct proof of the password failure. We should investigate it separately rather than changing executable permissions prematurely.

4. Next diagnostic: isolate pam_unix

I want to test whether Unix password authentication works independently of Howdy.

First run:

```
</> Bash

echo "===== PAM TESTER ====="
command -v pamtester || echo "pamtester not installed"

echo "===== ACCOUNT STATUS ====="
passwd -S remac

echo "===== ACCOUNT RECORD ====="
```

```
getent passwd remac

echo "===== XScreensaver AUTH BINARY ====="
command -v xscreensaver-auth
```

If `pamtester` is installed, create a temporary PAM service that bypasses Howdy:

```
</> Bash

cat > /etc/pam.d/howdy-unix-test <<'EOF'
auth required pam_unix.so nullok try_first_pass nodelay
account required pam_unix.so
EOF
```

Then run:

```
</> Bash

pamtester howdy-unix-test remac authenticate
```

Enter your normal account password when prompted. Do not put your password in the command itself.

If successful The underlying Unix password authentication works, and we can concentrate on the XScreenSaver/Howdy interaction.

If unsuccessful We investigate the underlying PAM password authentication before touching Howdy.

Afterward, remove the temporary test configuration:

```
</> Bash

rm -f /etc/pam.d/howdy-unix-test
```

5. What we are trying to prove

Component	Current finding
XScreenSaver PAM service	Working configuration
common-auth inclusion	Confirmed
Python Howdy module	Loaded in the stack

Face recognition	Not yet independently verified
pam_unix	Repeated authentication failures
Native graphical password conversation	Still under investigation
XScreenSaver OOM warning	Separate issue

My position: We should stop modifying the Python handler until the isolated Unix authentication test is complete. Otherwise, we risk introducing another variable into an already complicated authentication stack.

The objective is to fix the actual failure, not keep producing supposedly flawless versions of `pam_handler.py`.

2026-11-06 14:28:01 **You**

as root ┌──[Fri Nov 06 2:25:41 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─[REMAC]# echo "===== PAM TESTER ====="
command -v pamtester || echo "pamtester not installed"

echo "===== ACCOUNT STATUS ====="
passwd -S remac

echo "===== ACCOUNT RECORD ====="
getent passwd remac

echo "===== XScreensaver AUTH BINARY ====="
command -v xscreensaver-auth
===== PAM TESTER =====
pamtester not installed
===== ACCOUNT STATUS =====
remac P 2024-12-09 0 99999 7 -1
===== ACCOUNT RECORD =====
remac:x:1003:1003:,16,,:/home/remac:/usr/bin/zsh
===== XScreensaver AUTH BINARY =====
┌──[Fri Nov 06 2:25:46 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─[REMAC]# cat > /etc/pam.d/howdy-unix-test <<'EOF'

```
auth required pam_unix.so nullok try_first_pass nodelay
account required pam_unix.so
EOF
 ┌──[Fri Nov 06 2:25:50 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
 └──────[REMAC]# pamtester howdy-unix-test remac authenticate
zsh: command not found: pamtester
 ┌──[Fri Nov 06 2:26:05 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
 └──────[REMAC]# apt install pamtester
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package pamtester
 ┌──[Fri Nov 06 2:27:02 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
 └──────[REMAC]# dpkg -i /home/remac/Downloads/pamtester_0.1.2-4_amd64.deb
Selecting previously unselected package pamtester.
(Reading database ... 365625 files and directories currently installed.)
Preparing to unpack .../pamtester_0.1.2-4_amd64.deb ...
Unpacking pamtester (0.1.2-4) ...
Setting up pamtester (0.1.2-4) ...
Processing triggers for man-db (2.11.2-2) ...
 ┌──[Fri Nov 06 2:27:20 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
 └──────[REMAC]# pamtester howdy-unix-test remac authenticate
Password:
pamtester: successfully authenticated
 ┌──[Fri Nov 06 2:27:47 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
 └──────[REMAC]#

 and as me  ┌──[Fri Nov 06 2:27:30 PM][remac@Intel5]—[~]
 └──────[REMAC]$ pamtester howdy-unix-test remac authenticate
```

Password:
pamtester: Authentication service cannot retrieve authentication info
┌──[Fri Nov 06 2:27:39 PM][remac@Intel5]—[~]
└─[REMAC]$ pamtester howdy-unix-test remac authenticate
Password:
pamtester: Authentication service cannot retrieve authentication info
┌──[Fri Nov 06 2:27:58 PM][remac@Intel5]—[~]
└─[REMAC]$

ChatGPT gpt-6

This is a significant breakthrough. We have now established that **the correct password works through `pam_unix.so` when the authentication test runs as root.**

But the same test fails when executed as `remac`:

Test	Result
Root → `pamtester` → `pam_unix`	Successful
remac → `pamtester` → `pam_unix`	Authentication information unavailable

This changes our investigation considerably.

1. What this tells us

Your account is not showing as locked:

```
Plain text
remac P 2024-12-09 0 99999 7 -1
```

The password itself has also been verified successfully through the root-run test.

The difference is the execution privilege.

When root runs the test, `pam_unix` can access the protected authentication information directly. When an ordinary user runs it, PAM must use the appropriate privileged authentication helper.

This makes the permissions and execution configuration of `unix_chkpwd` particularly important.

There is also a second clue: your earlier XScreenSaver journal explicitly reported that `xscreensaver-auth` was not setuid root.

We should investigate both before changing your PAM stack.

2. Run these diagnostics as root

We need to inspect the actual privileged helper rather than change XScreenSaver permissions based on the warning.

```bash
echo "===== UNIX_CHKPWD LOCATION ====="
find /usr -name unix_chkpwd -o -name xscreensaver-auth 2>/dev/null

echo "===== UNIX_CHKPWD PERMISSIONS ====="
ls -l /usr/sbin/unix_chkpwd /sbin/unix_chkpwd 2>/dev/null

echo "===== SHADOW PERMISSIONS ====="
ls -l /etc/shadow

echo "===== FILE OWNERSHIP ====="
dpkg -S /usr/sbin/unix_chkpwd 2>/dev/null
dpkg -S /usr/libexec/xscreensaver/xscreensaver-auth 2>/dev/null

echo "===== MOUNT OPTIONS ====="
findmnt -T /usr/sbin/unix_chkpwd -o TARGET,OPTIONS
```

Also inspect the authentication errors generated by the non-root test:

```bash
journalctl -b --since "5 minutes ago" --no-pager |
grep -iE 'pamtester|howdy-unix-test|unix_chkpwd|pam_unix'
```

3. Important clarification about xscreensaver-auth

I investigated the warning against the XScreenSaver source code.

The setuid-root recommendation in that warning concerns the ability to adjust the parent process's OOM score. It is not proof that password authentication requires `xscreensaver-auth` to be setuid root. Debian maintainers have explicitly discussed leaving that permission decision to administrators.

Debian Sou… +2

Do not run `chmod 4755` on xscreensaver-auth. We have not established that it is necessary, and it would introduce additional privileged execution.

4. Our next decision

The results will help distinguish these possibilities:

- `unix_chkpwd` has incorrect ownership or permissions.
- The filesystem has `nosuid`, preventing the helper from obtaining its required privileges.
- PAM cannot retrieve the authentication information under the ordinary user's execution context.
- There is another PAM-specific restriction.

One particularly important distinction: your root-run test proves that `pam_unix` can authenticate the account when running with root privileges. It does not yet prove that the ordinary XScreenSaver authentication process can do the same.

Send me these diagnostic outputs. We are narrowing this down to a specific authentication boundary instead of guessing at Python changes.

2026-11-06 14:29:00 **You**

```
┌──[Fri Nov 06 2:28:42 PM][remac@Intel5]─[~]
└──╼[REMAC]$ echo "===== UNIX_CHKPWD LOCATION ====="
find /usr -name unix_chkpwd -o -name xscreensaver-auth
2>/dev/null

echo "===== UNIX_CHKPWD PERMISSIONS ====="
ls -l /usr/sbin/unix_chkpwd /sbin/unix_chkpwd 2>/dev/null

echo "===== SHADOW PERMISSIONS ====="
ls -l /etc/shadow

echo "===== FILE OWNERSHIP ====="
dpkg -S /usr/sbin/unix_chkpwd 2>/dev/null
dpkg -S /usr/libexec/xscreensaver/xscreensaver-auth 2>/dev/null

echo "===== MOUNT OPTIONS ====="
findmnt -T /usr/sbin/unix_chkpwd -o TARGET,OPTIONS
===== UNIX_CHKPWD LOCATION =====
/usr/libexec/xscreensaver/xscreensaver-auth
/usr/sbin/unix_chkpwd
===== UNIX_CHKPWD PERMISSIONS =====
```

```
-rwxr-xr-x 1 root root 39160 Aug 10  2025 /sbin/unix_chkpwd
-rwxr-xr-x 1 root root 39160 Aug 10  2025 /usr/sbin/unix_chkpwd
===== SHADOW PERMISSIONS =====
-rw-r----- 1 root shadow 1514 Nov  6 12:06 /etc/shadow
===== FILE OWNERSHIP =====
xscreensaver: /usr/libexec/xscreensaver/xscreensaver-auth
===== MOUNT OPTIONS =====
TARGET OPTIONS
/      rw,noatime
 ┌──[Fri Nov 06 2:28:50 PM][remac@Intel5]─[~]
 └──╼[REMAC]$ journalctl -b --since "5 minutes ago" --no-pager |
grep -iE 'pamtester|howdy-unix-test|unix_chkpwd|pam_unix'
Nov 06 14:27:31 Intel5 unix_chkpwd[73330]: check pass; user
unknown
Nov 06 14:27:33 Intel5 unix_chkpwd[73341]: check pass; user
unknown
Nov 06 14:27:33 Intel5 unix_chkpwd[73341]: password check failed
for user (remac)
Nov 06 14:27:33 Intel5 pamtester[73328]: pam_unix(howdy-unix-
test:auth): authentication failure; logname= uid=1003 euid=1003
tty= ruser= rhost=  user=remac
Nov 06 14:27:39 Intel5 unix_chkpwd[73377]: check pass; user
unknown
Nov 06 14:27:42 Intel5 unix_chkpwd[73389]: check pass; user
unknown
Nov 06 14:27:42 Intel5 unix_chkpwd[73389]: password check failed
for user (remac)
Nov 06 14:27:42 Intel5 pamtester[73376]: pam_unix(howdy-unix-
test:auth): authentication failure; logname= uid=1003 euid=1003
tty= ruser= rhost=  user=remac
 ┌──[Fri Nov 06 2:28:55 PM][remac@Intel5]─[~]
 └──╼[REMAC]$
```

ChatGPT gpt-6

We have found a much stronger lead. Your latest output points to a **missing privilege on** `unix_chkpwd`, rather than a defective password or an incorrect Howdy password-handling algorithm.

Look at these two results:

```
-rwxr-xr-x 1 root root 39160 /usr/sbin/unix_chkpwd
-rw-r----- 1 root shadow 1514 /etc/shadow
```

The helper is owned by root, but it has neither the setuid bit nor membership in the shadow group.

The Linux-PAM documentation states that `unix_chkpwd` is typically installed setuid root or setgid shadow. Linux manu... +1

That explains the difference:

- Root can read `/etc/shadow` and authenticate successfully.
- An ordinary user cannot read `/etc/shadow`.
- The helper currently has no apparent mechanism to obtain the necessary privilege.
- Password verification consequently fails in the ordinary-user context.

This also fits the XScreenSaver failure, where the authentication process runs with the ordinary user's identity.

1. Verify the package's expected permissions before repairing

I don't want us to blindly change a security-sensitive executable. First, run the following **as root**:

```bash
echo "===== PACKAGE OWNER ====="
dpkg-query -S /usr/sbin/unix_chkpwd
dpkg-query -S /sbin/unix_chkpwd

echo "===== INSTALLED PAM PACKAGES ====="
dpkg-query -W -f='${binary:Package} ${Version}\n' \
    libpam-modules libpam-modules-bin libpam-runtime

echo "===== PACKAGE VERIFICATION ====="
dpkg -V libpam-modules-bin

echo "===== PACKAGE OVERRIDES ====="
dpkg-statoverride --list | grep unix_chkpwd

echo "===== REAL FILE DETAILS ====="
readlink -f /usr/sbin/unix_chkpwd
stat -c '%A %a %U:%G %n' /usr/sbin/unix_chkpwd

echo "===== CAPABILITIES ====="
```

```
getcap /usr/sbin/unix_chkpwd
```

2. What happens next

If package metadata confirms that the helper should have setuid-root or setgid-shadow permissions, we can restore its intended permissions using the package's supported mechanism.

We should not invent permissions or grant broader privileges than necessary.

After that, we will repeat:

```
</> Bash

pamtester howdy-unix-test remac authenticate
```

First as `remac`, then test XScreenSaver.

My current leading diagnosis: the missing privilege on `unix_chkpwd` explains the root-versus-user authentication discrepancy and is a strong candidate for the underlying XScreenSaver password failure.

Let's verify the package metadata before making the repair. We may have found the actual root cause.

2026-11-06 14:30:08 **You**

```
┌──[Fri Nov 06 2:29:35 PM]
[root@Intel5]─[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────────[REMAC]# echo "===== PACKAGE OWNER ====="
dpkg-query -S /usr/sbin/unix_chkpwd
dpkg-query -S /sbin/unix_chkpwd

echo "===== INSTALLED PAM PACKAGES ====="
dpkg-query -W -f='${binary:Package} ${Version}\n' \
  libpam-modules libpam-modules-bin libpam-runtime

echo "===== PACKAGE VERIFICATION ====="
dpkg -V libpam-modules-bin

echo "===== PACKAGE OVERRIDES ====="
dpkg-statoverride --list | grep unix_chkpwd
```

```
echo "===== REAL FILE DETAILS ====="
readlink -f /usr/sbin/unix_chkpwd
stat -c '%A %a %U:%G %n' /usr/sbin/unix_chkpwd

echo "===== CAPABILITIES ====="
getcap /usr/sbin/unix_chkpwd
===== PACKAGE OWNER =====
dpkg-query: no path found matching pattern
/usr/sbin/unix_chkpwd
libpam-modules-bin: /sbin/unix_chkpwd
===== INSTALLED PAM PACKAGES =====
libpam-modules:amd64 1.5.2-6+deb12u2
libpam-modules-bin 1.5.2-6+deb12u2
libpam-runtime 1.5.2-6+deb12u2
===== PACKAGE VERIFICATION =====
===== PACKAGE OVERRIDES =====
===== REAL FILE DETAILS =====
/usr/sbin/unix_chkpwd
-rwxr-xr-x 755 root:root /usr/sbin/unix_chkpwd
===== CAPABILITIES =====
 ┌──[Fri Nov 06 2:29:42 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-
tricks/advanced-face-unlock-final/stable-version]
 └─────╼[REMAC]# pamtester howdy-unix-test remac authenticate
Password:
pamtester: successfully authenticated
 ┌──[Fri Nov 06 2:29:58 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-
tricks/advanced-face-unlock-final/stable-version]
 └─────╼[REMAC]#

  ┌──[Fri Nov 06 2:29:51 PM][remac@Intel5]—[~]
 └─────╼[REMAC]$ pamtester howdy-unix-test remac authenticate
Password:
pamtester: Authentication service cannot retrieve authentication
info
 ┌──[Fri Nov 06 2:30:05 PM][remac@Intel5]—[~]
 └─────╼[REMAC]$
```

We have now identified the likely root cause with much greater confidence.

Correction to my previous diagnosis: On Debian-based systems, `unix_chkpwd` can use the `setgid shadow` arrangement rather than `setuid root`.

The Debian package's documented installation mode is:

```
Plain text

-rwxr-sr-x root shadow /sbin/unix_chkpwd
```

That is mode `2755`, not `755`. Debian Ma... +2

Your installed file is:

```
Plain text

-rwxr-xr-x root root /usr/sbin/unix_chkpwd
```

That is the discrepancy we were looking for.

It also explains why:

- Root can authenticate.
- An ordinary user cannot retrieve the protected authentication information.
- XScreenSaver fails during password verification.

Your `dpkg -V` result does not rule this out: it does not establish that the installed file's permission bits match the package's intended security mode.

1. Repair the helper

As root, first verify that the shadow group exists:

```
Bash

getent group shadow
```

Then restore the expected ownership and permissions:

```bash
chown root:shadow /sbin/unix_chkpwd
chmod 2755 /sbin/unix_chkpwd
```

Verify:

```bash
stat -c '%A %a %U:%G %n' /sbin/unix_chkpwd
```

Expected:

```
-rwxr-sr-x 2755 root:shadow /sbin/unix_chkpwd
```

This restores the standard Debian privilege arrangement. Do not use `chmod 4755`; setgid-shadow is the arrangement indicated by the Debian package metadata.

2. Retest as remac

Switch to your ordinary account and run:

```bash
pamtester howdy-unix-test remac authenticate
```

Enter your normal password.

Expected `pamtester: successfully authenticated`

3. Retest XScreenSaver

If the independent test succeeds:

```bash
xscreensaver-command -lock
```

Test the following:

1. Enter your password normally.
2. Block the camera and try again.
3. Check whether the password remains in the input pipeline and authentication succeeds.

Keep your existing PAM configuration unchanged during this test.

Send me the ordinary-user `pamtester` result first. If it succeeds, we will have isolated and repaired the Unix authentication privilege problem. Then we can determine whether any remaining Howdy/PAM conversation issue needs a separate fix.

2026-11-06 14:31:28 **You**

```
┌──[Fri Nov 06 2:30:42 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────╼[REMAC]# getent group shadow
shadow:x:42:
┌──[Fri Nov 06 2:30:49 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────╼[REMAC]# chown root:shadow /sbin/unix_chkpwd
chmod 2755 /sbin/unix_chkpwd
┌──[Fri Nov 06 2:30:54 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────╼[REMAC]# stat -c '%A %a %U:%G %n' /sbin/unix_chkpwd
-rwxr-sr-x 2755 root:shadow /sbin/unix_chkpwd
┌──[Fri Nov 06 2:31:19 PM]
[root@Intel5]—[/home/remac/Desktop/youdemy/tips-and-tricks/advanced-face-unlock-final/stable-version]
└─────╼[REMAC]#



┌──[Fri Nov 06 2:31:03 PM][remac@Intel5]—[~]
└─────╼[REMAC]$ pamtester howdy-unix-test remac authenticate
Password:
pamtester: successfully authenticated
```

```
┌──[Fri Nov 06 2:31:09 PM][remac@Intel5]—[~]
└──╼[REMAC]$
```

ChatGPT gpt-6

Unix authentication problem resolved

Excellent. This is our first confirmed repair.

The ordinary-user test now succeeds:

```
</> Plain text

pamtester: successfully authenticated
```

Before the permission repair, the same test failed with:

```
</> Plain text

pamtester: Authentication service cannot retrieve authentication info
```

After restoring the helper to:

```
</> Plain text

-rwxr-sr-x 2755 root:shadow /sbin/unix_chkpwd
```

authentication works as `remac`.

We have confirmed that the missing setgid-shadow permission was responsible for the root-versus-ordinary-user authentication discrepancy.

This was an independent PAM authentication failure, not a Howdy camera-recognition problem.

Next: test the actual XScreenSaver lock screen

We have fixed the underlying Unix helper, but we have not yet confirmed that XScreenSaver now unlocks correctly.

Step 1. Remove the temporary test configuration

As root:

```bash
rm -f /etc/pam.d/howdy-unix-test
```

Step 2. Test native graphical authentication

As `remac`, from your active desktop terminal:

```bash
xscreensaver-command -lock
```

Test your normal password.

Step 3. Test face-recognition fallback

Repeat the lock test with the camera unavailable. Enter your password through the graphical dialog.

Step 4. Collect the result

After unlocking, run:

```bash
journalctl -b --since "3 minutes ago" --no-pager |
grep -iE 'xscreensaver-auth|pam_unix|pam_python|howdy|authentication'
```

Tell me two things:

1. Does the correct password now unlock XScreenSaver?
2. When the camera is blocked, does password authentication still work without clearing your input?

Important: We have fixed one confirmed defect. I don't want to declare the entire Howdy integration fixed until these graphical tests pass.

2026-11-06 14:34:15 **You**

yes password authenticated us └───────[REMAC]$ journalctl -b --since "3 minutes ago" --no-pager | grep -iE 'xscreensaver-auth|pam_unix|pam_python|howdy|authentication'
Nov 06 14:29:54 Intel5 pamtester[74052]: pam_unix(howdy-unix-test:auth): authentication failure; logname= uid=1003 euid=1003 tty= ruser= rhost= user=remac
┌──────[Fri Nov 06 2:32:41 PM][remac@Intel5]─[~]
└───────[REMAC]$

└───────[REMAC]$ journalctl -b --since "3 minutes ago" --no-pager | grep -iE 'xscreensaver-auth|pam_unix|pam_python|howdy|authentication'
Nov 06 14:29:54 Intel5 pamtester[74052]: pam_unix(howdy-unix-test:auth): authentication failure; logname= uid=1003 euid=1003 tty= ruser= rhost= user=remac
┌──────[Fri Nov 06 2:32:52 PM][remac@Intel5]─[~]
└───────[REMAC]$
so to add to my systemwide permissionfixer script what exact commands should i add to fix the issue chknx stuff

ChatGPT gpt-6

The confirmed fix was restoring the **setgid** `shadow` **permission on** `unix_chkpwd`. Add these commands to your system-wide permission fixer:

```bash
# Restore Debian PAM unix_chkpwd permissions
if [ -x /sbin/unix_chkpwd ]; then
    chown root:shadow /sbin/unix_chkpwd
    chmod 2755 /sbin/unix_chkpwd
fi
```

I recommend making it slightly more robust by checking the group exists and verifying the resulting permissions:

```bash
# Fix pam_unix password verification helper
if getent group shadow >/dev/null 2>&1 && [ -e /sbin/unix_chkpwd ]; then
    chown root:shadow /sbin/unix_chkpwd
```

```
    chmod 2755 /sbin/unix_chkpwd

    if [ "$(stat -c '%U:%G:%a' /sbin/unix_chkpwd)" = "root:shadow:2755" ]; then
        echo "[OK] unix_chkpwd permissions restored"
    else
        echo "[ERROR] unix_chkpwd permission verification failed"
    fi
else
    echo "[ERROR] shadow group or /sbin/unix_chkpwd not found"
fi
```

Expected permission state:

```
Plain text

-rwxr-sr-x root shadow /sbin/unix_chkpwd
```

The important details are:

- Owner: `root`
- Group: `shadow`
- Mode: `2755` — the leading `2` enables setgid.

Run and verify

As root, run:

```
Bash

chown root:shadow /sbin/unix_chkpwd
chmod 2755 /sbin/unix_chkpwd

stat -c '%A %a %U:%G %n' /sbin/unix_chkpwd
```

Expected:

```
Plain text

-rwxr-sr-x 2755 root:shadow /sbin/unix_chkpwd
```

One additional precaution: Debian package upgrades or permission-management tools could change this again, so keeping this check in your fixer is sensible. Don't apply `chmod 4755` to `xscreensaver-auth`; that is unrelated to the confirmed password-verification failure.